

TLI: Training-Free Lightweight Indexing for Long-Context Sparse Attention

with Upper-Bound Block Screening and Far/Near Budget Partitioning

2026-09-30

Abstract

Extending the effective context length of large language models is bounded by KV-cache bandwidth: with a 131K context on an 8B model, the full KV read per decode step dominates compute and caps throughput. Existing solutions face a hard trade-off. Training-free sparse indexers rely on lower-bound score approximations and collapse on long-context retrieval (we measure Quest at 21 on RULER 16K multikey₃, versus 100 for FullKV); training-based indexers such as DSA introduce learnable parameters and a 1000-step warm-up over 2.1B tokens, and their selection quality depends on the training distribution. This paper proposes TLI, a training-free two-level sparse attention indexer. The first level scores 64-token blocks with a minmax **upper bound** over a 4bit-quantized position-stable subspace, which guarantees no miss within the subspace-score protocol; the second level partitions the token budget into a far pool and a near pool so that distant retrieval-critical tokens survive competition with near high scorers. On 13 LongBench tasks with Qwen3-8B, TLI scores 50.54, matching FullKV (50.36) within noise; on the far-heavy multi-hop task musique it reaches 34.76, the highest score among all methods including FullKV. On RULER it reaches 87.93 against 88.71 for FullKV, and the gain of the partition over the single-pool ablation arm turns positive only at 16K (-0.14 and -0.25 at 4K and 8K, $+0.91$ at 16K), concentrated on multi-distractor retrieval and common-word tasks. On the speed side, the TLI selection chain achieves a kernel-level speedup of $1.46\times$ at 32K to $5.09\times$ at 128K over dense scoring, and $1.285\times$ end-to-end for a single 64K request; all accuracy costs and negative results are reported under matched hardware and matched protocol. TLI ships as an integration in sglang in production form (paged addressing, incremental indexing, CUDA graph).

1 Introduction

The ability to process long context is becoming a core competitive capability of large language models. Long-chain reasoning in DeepSeek-R1 [1], million-token context products from Kimi and Claude, and long-document question answering and repository-scale code understanding all require a model to consume tens of thousands to hundreds of thousands of tokens within a single request. Every extension of the context directly inflates the KV cache: with a 131K context on an 8B model, the KV read already dominates per-step compute, so the full KV access in decode forms a bandwidth bottleneck, and prefill is likewise constrained by full-attention bandwidth. Inference-system advances (PagedAttention [4], FlashAttention [3]) optimize memory organization and compute kernels, but they do not reduce the amount of data that must be read.

The way to cut KV reads is attention sparsity: softmax normalization concentrates the overwhelming majority of attention mass on a small number of tokens. Our trace analysis on real data with Qwen3-8B gives the concrete shape of this distribution: the mass of the dense top-1024

selection decomposes into three layers, sink (0.37–0.71), a near sliding window (0.16–0.29), and distant retrieval (0.02–0.29), and the distant layer profile correlates across layers within a task at 0.93–1.00. This three-layer structure implies the following: a sparse indexer that preserves quality must simultaneously satisfy three kinds of attention with completely different properties, and their budget demands conflict with each other.

Existing solutions fall short along three paths. The first family, training-free sparse indexing, is represented by Quest [5], which constructs a score **lower bound** from per-page sampled minima. A lower bound underestimates genuinely high-scoring pages, and miss rates worsen as the number of distractor keys grows superlinearly with length; we measure this family collapsing to 21 on RULER 16K multikey₃ (FullKV scores 100). KV compression methods (SnapKV, H2O, PyramidKV [6, 7, 8]) discard KV once at prefill and cannot respond to the decode-side query distribution. The second family, training-based indexing, is represented by DSA [13] (DeepSeek V3.2), which trades a learnable projection and ReLU scoring for selection capability at the cost of a 1000-step warm-up and a 2.1B-token training budget, with selection quality tied to the training distribution. The third family, the two-level structure (HISA [15], COLM 2026), validates the block-coarse-plus-token-refine form, but two orthogonal design dimensions within it, whether the score representation takes an upper bound or a lower bound, and whether the budget is organized as a partition or a single pool, have not been systematically characterized for their quality and speed impact.

This raises a critical research question: can we design a training-free sparse attention indexer with zero warm-up that, without introducing any learnable parameter, simultaneously preserves long-context retrieval quality and delivers the bandwidth win?

This paper proposes TLI (Training-free Lightweight Indexer) to answer this question. TLI is a two-level cascaded indexer. The first level organizes the KV cache into 64-token blocks, maintains per-block min/max bounds over a position-stable low-frequency subspace ($d'=32$) stored in 4bit quantized form, and scores each query against the optimistic side of the block bound: the upper bound guarantees no miss, so the block containing any genuinely high-scoring token is always admitted. The second level splits the token budget into a far pool and a near pool: the near sliding window is force-covered for locality, and the far pool applies token-level fine selection to the blocks admitted by L1, with an independent quota so that distant retrieval-critical tokens are not evicted by near high scorers. A per-request perception gate closes the chain: one softmax statistic from the last prefill segment identifies layers whose far mass is near zero, and decode safely skips their far retrieval. The entire selection chain is kernelized as a fused L1 block scorer, an L2 cascaded dual topk, and a unified sparse attention kernel, and an $O(n)$ exact incremental index makes block-bound maintenance independent of sequence length.

Our contributions are as follows:

1. **A systematic characterization of training-free dimensionality-reduction freedom.**

The position-stable low-frequency subspace is lossless for indexing quality (mass coverage of the full-dimensional top-1024 at 0.729, versus 0.732 for full-dimensional scoring itself, against 0.32 for a random subspace). A triple verdict, measured in the far-region full-chain mass-recall protocol, confirms that the power of tail32 comes from **the 16 complete lowest-frequency RoPE rotation pairs**: frequency monotonicity (low-frequency pairs 0.811, mid-frequency 0.360, high-frequency 0.004), pairing completeness (a misaligned pairing collapses to 0.567), and saturation at 16 pairs (8 pairs reach only 0.473). A four-quadrant characterization of selection methods bounds the feasible region: per-dimension spectral selection (0.759) and trained projections both lose to pairing entanglement; query-adaptive pair selection (0.865) cannot be indexed; per-layer static pairs (0.849) are index-compatible, so tail32 is the zero-signal-cost optimum. Unsupervised PCA compresses fine selection further to $d=16$ at near-

loss (0.766 versus 0.804), and a shared SVD basis across kv-heads matches tail32 on 8/8 samples. Trained projections hold up in trace replay (92–94%) yet collapse end-to-end: the projection basis is fragile under decode query-distribution drift, whereas the rotation-pair frequency prior is independent of the query distribution and stays stable. Dimensionality-reduction freedom comes from position priors, not from learning or adaptation.

2. **Far/near budget partition.** The selection index is built per kv-head: query heads in the same group share one kv-head-level selection, with no aggregation needed. Under GQA the distant attention mass is extremely uneven across kv-heads (a single kv-head can hold 0.804), so single-pool top-k evicts distant retrieval-critical tokens in favor of near high scorers. The partitioned independent quota holds on both e2e benchmarks (LongBench +0.25 and RULER +0.17 over the single pool), with the purest gain on far-heavy multi-hop tasks (musique 34.76 versus 29.74 for the same-budget single-pool arm, +5.02). Cluster-representative selection shows no consistent advantage under a strict token budget and is demoted to a negative-result ablation.
3. **A per-request perception gate.** Static layer masks do not transfer across tasks (multi-hop tasks lose 4.8–5.9 points); a dynamic signal from the last prefill segment correlates with decode-side far quality at 0.924, and in the real triggering regime ($\tau=0.005$), the gate-on versus gate-off difference is +0.23 on musique and -0.41 on qasper, and no far-heavy task collapses. The gate is positioned as a conservative switch that saves compute safely, not as a source of accuracy gain.
4. **Tuning-free lightweight configuration selection.** The quality surface of the partition parameters $\alpha/\beta/\gamma$ is flat at both the task and the layer granularity: the per-task oracle of choosing them freely gains only +0.08 over the fixed default (13 tasks), the per-layer oracle only +0.0008, and per-request online dispatch driven by a one-shot prefill far-statistic nets zero. Deployment therefore needs one default configuration, with a single β increase as the only optional adjustment for far-heavy multi-hop tasks; the absence of tuning is itself a deployment advantage over the 1000-step warm-up of DSA.

2 Method

2.1 Preliminaries: Standard Attention and the Sparse Selection Problem

We first recall standard attention. A single query token $q \in \mathbb{R}^{1 \times d}$ attends to N KV tokens ($K, V \in \mathbb{R}^{N \times d}$):

$$\text{Attn}(q, K, V) = \text{Softmax}(qK^\top) V. \quad (1)$$

The sparse indexing problem is to select for q a set of positions $I \subseteq [N]$ of size K_2 such that $\text{Attn}(q, K[I], V[I])$ approximates the full result. We treat the single-head case for clarity; per-kv-head scoring under GQA and the cross-head unevenness of distant mass are discussed in the partition mechanism of Section 2.2. Unlike a training-based indexer, a training-free indexer admits no parameters: selection quality depends entirely on which signal scores candidates and how the budget is organized.

TLI: training-free lightweight indexing — upper-bound block screening with far/near budget partitioning

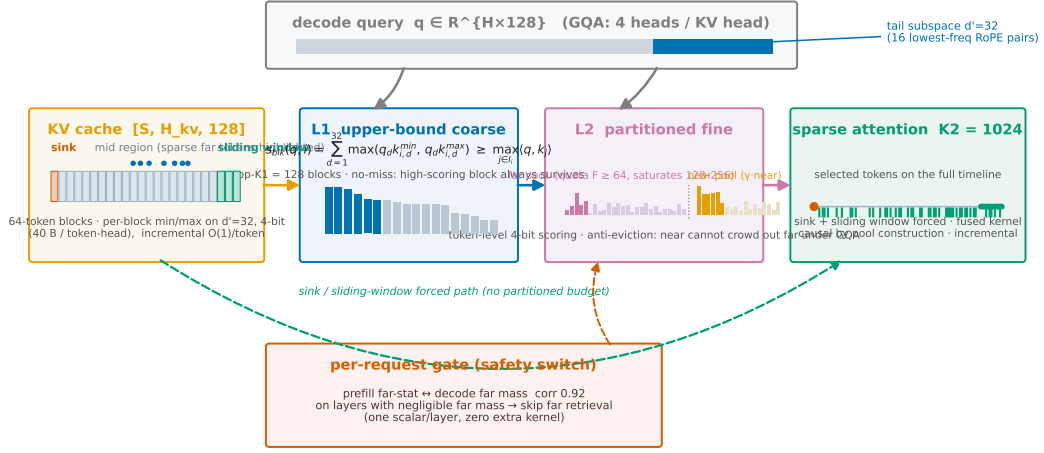


Figure 1: TLI two-level indexer architecture: L1 block-level minmax upper-bound coarse selection (subspace $d'=32$ with 4bit quantization) $\rightarrow$ L2 far/near partitioned token-level fine selection $\rightarrow$ a unified sparse attention fused kernel.

2.2 TLI Architecture

Different from standard attention, where every query reads the full context, TLI selects $K_2=1024$ KV tokens per query head:

$$\text{TLI}(q, K, V) = \text{Attn}(q, K[I_{L1 \rightarrow L2}], V[I_{L1 \rightarrow L2}]), \quad (2)$$

where $I_{L1 \rightarrow L2}$ is produced by a two-level cascade (Figure 1). The design contribution of TLI is not the cascade form itself (HISA [15] already validated it) but the choices made on two orthogonal dimensions: **the score representation takes an upper bound**, and **the budget organization takes a partition**.

L1: block-level upper-bound coarse selection. The context is split into blocks of $B=64$. For each block and each kv-head, TLI maintains min/max bounds over a position-stable low-frequency subspace ($d'=32$) in 4bit quantized storage (128 dimensions compressed to 40B/token-head). The query score of a block takes, per dimension, the bound endpoint matching the sign of q_d and sums these optimistic terms:

$$s_{\text{blk}}(q, i) = \sum_{d=1}^{d'} \max(q_d k_{i,d}^{\min}, q_d k_{i,d}^{\max}) \geq \max_{j \in I_i} \langle q, k_j \rangle. \quad (3)$$

This upper bound guarantees **no miss within the subspace-score protocol**: the block containing a genuinely high-scoring token scores at least as high as that token on the subspace and always enters the top- K_1 (the subspace itself is lossy with respect to the full dot product; that loss is characterized separately by the selection methodology in §2.3). This contrasts with the page-min lower bound of Quest [5], which underestimates genuinely high-scoring pages and misses more as distractors multiply. The top $K_1=128$ blocks proceed to L2.

L2: far/near partitioned fine selection. The sink and the sliding window (the most recent sw_{lo} tokens, whose locality is guaranteed by sliding-window semantics and needs no scoring) form the forced-retention region and consume no partitioned budget; the remaining mid budget splits

into two pools. The far pool applies token-level 4bit fine selection over the blocks admitted by L1, taking the far quota F (saturating at 128–256); the near pool fine-selects the near region under the same protocol. The anti-eviction mechanism of the partition is as follows. Under GQA the distant mass is extremely uneven across kv-heads (trace measurement: one head alone holds 0.804 while other heads in the same group are near zero), so in a single-pool top-k the far candidates compete against near high scorers in the same pool and the retrieval-critical tokens of far-heavy layers are systematically evicted. The partition gives far its own quota, and near-side deductions carry a near_floor guard against boundary regression.

Formal definition of the budget. Let the query prefix length be $S = t + 1$, the mid-region length be $L_{\text{mid}} = S - |I_{\text{sink}}| - sw_{lo}$, and the near-region length be $\ell_{\text{near}} = \lceil \alpha L_{\text{mid}} \rceil$. The two pools and the final selection set are

$$I_{\text{far}} = \text{TopK}_F \left\{ \langle q, k_j \rangle : j \in \bigcup_{i \in I_{L1}} I_i \cap [|I_{\text{sink}}|, S - sw_{lo} - \ell_{\text{near}}] \right\}, \quad (4)$$

$$I_{\text{near}} = \text{TopK}_{K_2^{\text{mid}} - F} \left\{ \langle q, k_j \rangle : j \in [S - sw_{lo} - \ell_{\text{near}}, S - sw_{lo}] \right\}, \quad (5)$$

$$I_{L1 \rightarrow L2} = I_{\text{sink}} \cup [S - sw_{lo}, t] \cup I_{\text{near}} \cup I_{\text{far}}, \quad (6)$$

where the mid budget is $K_2^{\text{mid}} = K_2 - |I_{\text{sink}}| - sw_{lo}$ and the far quota is $F = \max(64, K_2^{\text{mid}} - \lceil \gamma \lceil \ell_{\text{near}} / B \rceil B \rceil)$: γ converts the near block count into the near fine-selection quota, far takes the remaining budget with a floor of 64, and β acts on the near page budget at L1.

Two designs that preserve causality. An autoregressive language model must not let a query access future tokens. TLI preserves causality through two concrete designs. First, **pool boundaries are causal boundaries**: the far pool’s upper position bound is $\leq t + 1 - \ell_{\text{near}} - sw_{lo}$ and the near pool stays below sw_{lo} , so the union of the two pools falls inside the causal range by construction, with no explicit causal mask (garbage blocks from L1 also fall outside the pools). Second, **uniform coverage for early rows**: within the first prefill chunk, early rows whose position t_r is below K_2 lack candidates, and if sentinel or future positions returned by topk leak in, non-causal leakage propagates layer by layer (we observed repetitive-loop generation collapse on a 30B model with exactly this root cause; the 8B model tolerated the same bug without collapsing). Early rows use a uniform repeated grid, which is equivalent to dense.

Relation to sink and sliding-window attention. Sink tokens (StreamingLLM [9]) and sliding-window attention (Longformer, BigBird [10, 11]) are two established static sparse structures, and we adopt them as known components: sink tokens carry 0.37–0.71 of attention mass and the sliding window carries near locality, together forming the forced-retention region of TLI that does not consume the partitioned budget. Our contribution is not the retention region itself but **the allocation of the remaining budget between far retrieval and near fine selection**. As corroborating evidence, we measured in baseline adaptation that SnapKV, H2O, and PyramidKV without forced sink retention emit EOS as their first token on Qwen3-8B (the mean sink weight in the last layer is 0.592 while shallow-layer selection signals do not point to sink, a layer-by-layer progressive distortion): any sparse method that does not handle sink explicitly distorts on this model family.

2.3 Design Choice Discussion

We next discuss three key design choices of TLI.

Subspace choice. L1 takes the 32-dimensional low-frequency tail because the indexing signal must preserve the ranking power of full-dimensional scores after reduction. The rotate_half

RoPE of Qwen3 makes dot products over the low-frequency dimensions rank-consistent with full-dimensional dot products (mass coverage of the full-dimensional top-1024 at 0.729, versus 0.732 for full-dimensional scoring itself), while a random subspace collapses to 0.32 and high-frequency dimensions to 0.137. The mechanism is confirmed by a triple verdict (Figure 2): all 128 dimensions of Qwen3 participate in RoPE, and under the rotate.half layout frequency j acts on the dimension pair $(j, j+64)$, so tail32 (dimensions 48..63 and 112..127) equals **the 16 complete lowest-frequency rotation pairs**. The dot product decomposes by frequency as $\sum_j [(q_j k_j + q_{j+64} k_{j+64}) \cos(\Delta \omega_j) + (q_j k_{j+64} - q_{j+64} k_j) \sin(\Delta \omega_j)]$: at large distant Δ the low-frequency terms are smooth and highly correlated while the high-frequency terms oscillate and cancel, so the low-frequency pairs carry the long-range retrieval signal. Three verdicts support this structural account: frequency monotonicity (low-frequency 16 pairs 0.811, mid-frequency 0.360, high-frequency 0.004); pairing completeness (a misaligned pairing, taking the low-frequency 16 from the first half but a mid-frequency source for the second half, collapses to 0.567 with nearly identical segment positions); and saturation at 16 pairs (16 pairs reach 0.811, near-saturated, while 8 pairs reach only 0.473). The direction replicates on the 32B model (complete pairs 0.829 > misaligned 0.645 > single half 0.54), with the saturation width shifting to 24 pairs (0.904).

The full characterization of selection methodology (Table 1): crossing static (index-compatible) with adaptive (per-query) selection yields four quadrants. Per-dimension spectral top-32 reaches only 0.759: the two elements of a rotation pair are entangled in the dot product, so per-dimension importance cannot in principle recover the pairing structure. SparQ-style [12] per-query $|q|$ -magnitude top-32 reaches 0.848, and pair-aware adaptive selection (ranking complete pairs by $|q_j| + |q_{j+64}|$) reaches 0.865. Adaptive selection does yield a small gain, but its subspace changes per query, which is structurally incompatible with an incremental minmax block index built before the query arrives (SparQ pays a per-query dimension-gather cost for exactly this reason and builds no index). The intermediate point is per-layer static selection: the top-16 complete pairs by pair magnitude from the q statistics of the last prefill segment, fixed per layer and indexable, at 0.849, which keeps seven tenths of the adaptive gain at 3.8 points above tail32. **tail32 is precisely the optimum of the zero-signal-cost feasible region**, not the global optimum; static pair selection is the indexable upgrade path. Trained projections rank more accurately (92–94% in trace replay) yet collapse end-to-end: the projection basis is fragile under decode query-distribution drift, whereas the rotation-pair frequency prior is independent of the query distribution and stays stable. This protocol gap, where the trace protocol holds and the e2e protocol collapses, is an empirical characterization of the dividing line between the training-free and the training-based families.

Table 1: Four quadrants of subspace selection methodology (far full-chain mass recall, mean over 8 samples).

Selection method	Signal	recall	Index-compatible
Per-dim spectral top-32	data-driven (static)	0.759	compatible
tail32	frequency prior (static)	0.811	compatible
SparQ-style $ q $ top-32	query-adaptive	0.848	incompatible
Pair-magnitude top-16 pairs	query-adaptive	0.865	incompatible
Per-layer static pairs	prefill q statistics	0.849	compatible

Budget partition parameters. The budget space is described by three parameters: $\alpha=0.125$ (near-region fraction), $\beta=0.375$ (near page budget), and $\gamma=0.125$ (near token budget), with far/near methods set to (minmax, avg). A grid of 132 configuration points plus five method-combination e2e arms yields three regularities: α is monotone toward its boundary (the less far budget the

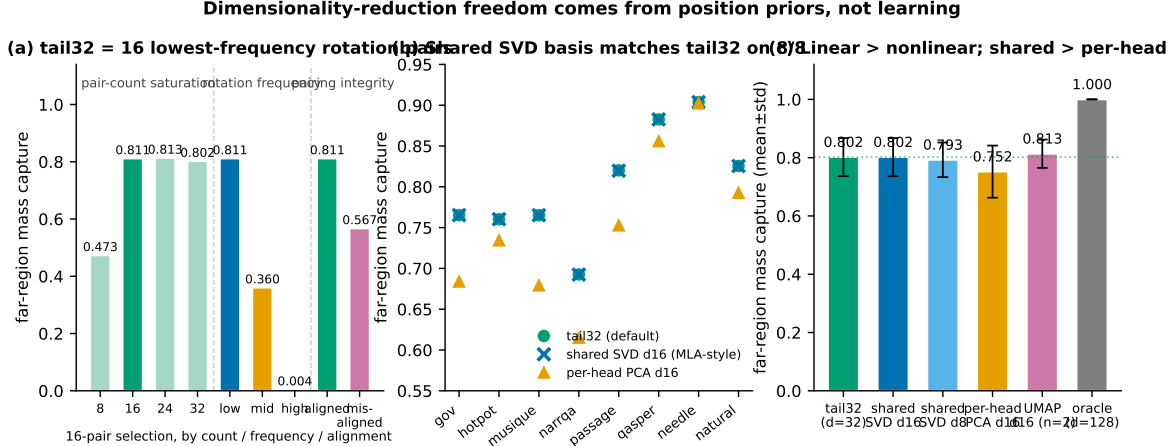


Figure 2: Dimensionality-reduction freedom comes from position priors: (a) tail32 equals the 16 complete lowest-frequency rotation pairs (frequency monotonicity, misaligned pairing collapses to 0.567, saturation at 16 pairs); (b) a shared SVD basis at $d=16$ matches tail32 on 8/8 samples; (c) linear beats nonlinear, shared beats per-head.

better, limited by near_floor); β is flat over 0.25–0.75 (multi-hop tasks favor 0.375); and γ must scale proportionally with K_2 (the lesson of $\gamma=0.5$ under $K_2=1024$, where far retains only the floor quota of 64 and F1 collapses to 13.51). The most important finding is the flatness itself: the per-task oracle that perfectly selects β gains only +0.08, and the per-layer oracle gains only +0.0008, so **the budget partition is insensitive to configuration error and needs no tuning**. This is the deployment-simplicity argument: against the 1000-step warm-up of DSA, one fixed TLI configuration covers 12/13 tasks.

Gate threshold. The dynamic gate uses last-segment prefill far statistics to identify layers whose far mass is near zero. The verdict on the threshold τ comes from a two-task gradient: at $\tau=0.005$, musique moves +0.23 and qasper -0.41 ; at $\tau=0.015$, +0.27 and -0.67 . The gradient is monotone, and the real triggering regime takes small losses without collapsing (against 4.8–5.9 lost by the static variant). The gate is therefore positioned as a conservative switch that saves compute safely.

2.4 Implementation

We implement TLI in the production form of slang; the selection chain consists of four kernel components:

- **Fused L1 block scoring:** one Triton kernel performs gather, 4bit dequantization (shared scale table), GEMV scoring, and block-level reduction, $3.6\times$ on the 32K shape;
- **L2 cascaded dual kernel:** a single launch performs far/near dual-pool scoring and partitioned topk, with pool boundaries doubling as causal boundaries (replacing the eager fine-matrix causal mask);
- **Unified sparse attention fused kernel:** gather, online softmax, and bf16 writeback, $11\text{--}16\times$ at the kernel level;
- **O(n) exact incremental indexing:** per-token incremental block-bound updates match full rebuild bit for bit, at 0.128ms independent of sequence length.

Three system-side pieces connect the design to production: paged addressing (selection outputs logical positions resolved through the req.to.token indirection into the KV pool, zero-copy); a per-layer shared, pre-allocated index pool (the launch count of batched decode selection is independent of batch size); and a CUDA-graph capture region with zero host synchronization. The design constraints come from the H20 profile: Tensor Core throughput is only 15% of H100 while HBM bandwidth is in the same class, so the first-order constraint on scoring kernels is bandwidth rather than compute. We measured and rejected both the Tensor Core path and the TMA transposed layout (arithmetic-intensity profiling shows the pure gather path is already bandwidth-saturated), so the kernel line of attack is eliminating intermediate materialization and merging launches.

3 Experiments

3.1 End-to-End Quality

This section validates the quality of TLI on two benchmarks: RULER [16] (3 lengths $\times$ 11 tasks $\times$ n=100, NVIDIA’s official pre-generated set) probes long-context retrieval, and 13 LongBench [17] subsets probe the mixed regime of real tasks. Among the compared methods, TIA is the first-generation training-free block upper-bound indexer built on the same pipeline as ours (block min/max upper bounds with 4bit subspace quantization, no partitioned budget and no perception gate) and serves as the most direct structural ablation reference for TLI; Quest represents the lower-bound family; the three KV-compression baselines run in the unified monkeypatch pipeline. All experiments use the real weights of Qwen3-8B [2], and TLI uses its final configuration (the strict protocol in which sink and sliding-window forced-retention tokens consume no budget).

Table 2: RULER four-method final table (string_match_all $\times$ 100; TLI uses the partitioned configuration: (minmax, avg) scoring with $\alpha=0.125$, $\beta=0.25$, $\gamma=0.125$, and $K_2=1024$).

	FullKV	TIA@1024	TLI@1024	Quest@1024
4K	91.59	91.63	91.49	87.91
8K	89.14	88.90	88.62	81.76
16K	85.41	84.53	83.68	70.18
Overall AVG	88.71	88.35	87.93	79.95

Three observations hold as shown in the table. First, the lead of TLI over Quest grows super-linearly with length (+3.58 $\rightarrow$ +6.86 $\rightarrow$ +13.50): at 16K, Quest collapses on the multi-distractor retrieval tasks while TLI stays within the correct order of magnitude, because lower-bound misses worsen as distractors multiply whereas upper-bound coarse selection has no such degradation. Second, the gain of the partition over the single-pool ablation arm turns positive only at 16K (−0.14 $\rightarrow$ −0.25 $\rightarrow$ +0.91, all measured against the same-configuration single-pool arm): ultra-long context is the payoff regime of partitioned far retrieval, and the 16K partition gain concentrates in multikey_3 (+5.0) and cwe (+5.4). Third, absolute scores must be read against FullKV at the same length: aggregate enumeration tasks degrade for FullKV itself as length grows (multivalue 89.5 $\rightarrow$ 69.25 $\rightarrow$ 44.25 is the enumeration ceiling of the model), so the same-length gap is the sparse effect.

On the 13 LongBench subsets (Table 3), TLI scores **50.54**, level with FullKV (50.36, +0.18 at the edge of noise, as flagged in Limitations), and +2.82 over Quest. The structure of the gain is clean: on the far-heavy multi-hop task musique, TLI reaches 34.76, the highest among all methods (+2.62 over FullKV, +5.02 over the same-budget single-pool arm), which is direct evidence of

Table 3: LongBench 13-subset final table (official per-task metric averaged; the three KV-compression baselines run in the unified monkeypatch pipeline at a 1024 budget, without sink-guard adaptation).

	FullKV	Quest	TIA	TLI	SnapKV	H2O	PyramidKV
Overall	50.36	47.72	50.06	50.54	27.70	14.87	28.82
musique	32.14	27.09	32.28	34.76	8.61	0.63	8.99

partitioned far retrieval; the remaining 12 tasks trade small wins and losses within noise (0 to -0.44). The full collapse of the three KV-compression baselines confirms the sink claim: SnapKV lands at 27.70 and H2O collapses further (musique 0.63, hotpotqa 2.68), because without an explicit sink guard Qwen3-8B emits EOS as its first token (mechanism at the end of §2.2); only the code tasks survive partially (lcc 67.19 and 54.06). We report the task-profile dependence of the budget mix honestly: LongBench multi-hop tasks favor $\beta=0.375$ (the 50.54 row), while RULER synthetic needle tasks favor $\beta=0.25$ (the 87.93 row), so both rows are reported and practitioners select by task profile. The per-task-family decomposition gives the boundary criterion for the β concession: it holds across all real-task families (musique, the most far-heavy task, gains the most at +1.94) and fails on the synthetic far-dense family (multikey_3 -2.00 , where the exploding far-candidate count makes budget granularity dominate): far-dense retrieval is the hard region for the entire sparse-indexer family (Quest collapses on the same task with the same structure), not a defect specific to the β mix.

3.2 Ablation

Subspace (A). The low-frequency $d'=32$ reaches mass coverage of the full-dimensional top-1024 at 0.729, versus 0.732 for full-dimensional scoring itself (random 0.32, high-frequency 0.137), so the position-stable subspace exists. The mechanism is confirmed by the triple verdict: tail32 equals the 16 complete lowest-frequency rotation pairs (frequency monotonicity 0.811/0.360/0.004, misaligned pairing collapses to 0.567, saturation at 16 pairs). The four-quadrant characterization of selection methods (Table 1) bounds the feasible region: per-dimension spectral selection at 0.759 and trained projections both lose to pairing entanglement, query-adaptive pair selection at 0.865 is the oracle ceiling but cannot be indexed, per-layer static pairs at 0.849 are index-compatible, and tail32 is the zero-signal-cost optimum. The dimensionality-reduction verdict has three parts: unsupervised PCA at $d=16$ is near-lossless (0.766 versus 0.804) and is the only recommended option; trained projections hold in trace replay and collapse in e2e (the protocol gap); and nonlinear methods (UMAP, t-SNE) add no quality while costing 80–167s per layer-sample to fit versus 0.128ms per token for incremental indexing, four orders of magnitude and therefore infeasible. A shared SVD basis across kv-heads at $d=16$ matches tail32 on 8/8 samples and beats per-head bases, so GQA head redundancy makes the training-free proxy of an MLA-style single-projection cache viable.

Partition. Token-level fine selection in the far region approaches the oracle (0.999 on far-heavy layers); cluster representatives show no consistent advantage under a strict token budget (block scatter-amax is the worst at 0.09–0.39), and cluster centers are not upper bounds, which leaves a theoretical miss hole, so clustering is demoted to a negative result. Partition versus single pool holds on both e2e benchmarks (LongBench +0.25, RULER +0.17), and its disagreement with the trace-replay protocol (the single pool wins slightly at wide budgets) pins down the mechanism: **the partition gain is budget allocation under a tight budget, not candidate quality.**

Layer skipping (D'). The offline mean profile skips 13/36 layers (precision 0.92–1.00), but a

global static mask does not transfer across tasks (multi-hop tasks lose 4.8–5.9); the dynamic prefill signal correlates at 0.924 and blocks the reverse error, and the real triggering regime takes small losses without collapsing, so the gate is a safety switch.

The rejection of adaptive α/β . Per-request online dispatch of α/β is rejected: the oracle ceiling is +0.08 on LongBench and +0.00 on RULER, and the +0.08 is fully explained by positive noise-selection bias (the median $|\Delta|$ over the 12 non-musique tasks is 0.17). The signal is available at zero cost, but the flat parameter surface leaves no leverage for adaptation. The best use of the same signal is the gate (safety value), not β dispatch (no accuracy value). Refining the granularity to layers adds no leverage either (per-layer oracle gain averages +0.0008), so both granularities, task and layer, reject adaptation, and the finding translates into the deployment-simplicity argument.

The protocol gap (methodology). Trace-replay mass rankings and e2e F1 rankings systematically reverse across the five method-combination arms (the near-average-scoring arm, ranked 4th in replay, is the full e2e champion at 50.54): offline screening cannot replace the e2e verdict, and both protocols must be measured and both reported.

3.3 Kernel-Level Microbenchmarks

The selection chain achieves the following speedups over dense full-dimensional scoring: $1.46\times$ at 32K, $2.77\times$ at 64K, and $5.09\times$ at 128K (synthetic data, labeled as such, trace-replay protocol). The same-machine three-way comparison (official kernels imported as-is into a unified harness, 131K with cudaEvent timing) gives Quest 0.107ms, DSA 0.503ms, and TLI 0.787ms. The latency ranking is Quest < DSA < TLI, which we report as measured; the current advantage of TLI sits on the algorithm-structure side: ≈ 258 MACs per token for indexing (1/32 of DSA), index storage $3\times$ below Quest, and quality versus Quest at +2.82 on LongBench and +7.98 on RULER. At 131K all three sit far from the HBM bound, so the ranking reflects implementation maturity, not the algorithmic ceiling.

3.4 End-to-End Speed

The e2e verdict (Qwen3-30B-A3B, $2\times H20$): a single 64K request reaches $1.285\times$ (16.39s versus 21.06s for dense), the first net cash-out of the theoretical sparse-traffic win at e2e; the 32K shape reaches $0.77\times$ (fixed overhead not yet covered at short context); the TP2 bs16 64K shape reaches $0.92\times$, and we attribute the remaining 8% as measured to launch-bound behavior (the per-request Python loop leaves the GPU nominally saturated while bandwidth utilization is only 10–11%), with cross-request batching already validated at $3.8\times$ as the fix. The 8B steady state (bs16 $\times$ S=30K $\times$ n=256): prefill 733.9 $\rightarrow$ 183.4s ($4.00\times$) and decode step 65.0 $\rightarrow$ 33.5ms ($1.94\times$, overtaking dense at 40.4ms).

3.5 Profiling Methodology Findings

Per-kernel microbenchmarks and production e2e differ systematically: the three host synchronizations on the select hot path (.item(), .any()) cause $\sim 100K$ queue drains in production form. A synthetic bench without a GPU queue measures none of this, and removing the synchronizations improves prefill by 6.7% on the same workload. Synchronization-elimination optimizations must be validated at the e2e layer. We combine this with the observation that offline trace-replay rankings cannot substitute for e2e verdicts into one methodology: measure and honestly report both the kernel level and the e2e level.

4 Related Work

Sparse attention indexing. Quest [5] constructs a lower-bound approximation from page sampling, and the KV compression family (SnapKV, H2O, PyramidKV [6, 7, 8]) discards KV once at prefill. TLI differs from them on three counts: the score representation takes an upper bound (versus a lower bound: the upper-bound family leads Quest by +8.40 on the overall RULER average, the dominant source of the long-context quality split), the budget organization takes a partition (versus a single pool), and the structure is a two-level cascade (versus a single level). While adapting SnapKV, H2O, and PyramidKV under a unified protocol, we found that implementations without forced sink retention emit EOS as their first token on Qwen3-8B: sink receives no selection signal in shallow layers yet carries 0.59 of attention mass in deep layers, a layer-by-layer progressive distortion. This is both a fairness correction for adapting this family of baselines and corroborating evidence for forced sink retention.

Training-based indexing. DSA [13] (DeepSeek V3.2) trains a selector with $I = \sum w \cdot \text{ReLU}(q \cdot k)$, 64 heads $\times$ 128 dimensions in FP8 with top-2048, and a 1000-step warm-up; MoBA [14] applies gated block sparsity. TLI is training-free and weight-free, and we compare against them in a unified kernel-level harness. The two form a methodological divide: DSA trades training for selection capability and can learn arbitrary projections, while a training-free method can rely only on position priors that are independent of the query distribution, and the protocol gap (the trace protocol holds while e2e collapses) is the empirical characterization of this dividing line.

Two-level structures. HISA [15] (COLM 2026) validates the block-coarse-plus-token-refine form, so the two-level form itself is not our contribution. The increment of TLI is the systematic characterization of two orthogonal dimensions: representation (upper bound versus lower bound) and budget organization (partition versus single pool), and their quality-speed impact.

Inference systems. The FlashAttention [3] line optimizes compute kernels, and PagedAttention/vLLM [4] optimizes memory organization. TLI is orthogonal to both and integrates into sglang in production form.

5 Limitations and Future Work

On accuracy, the margin of the first LongBench win over FullKV sits at the edge of noise (+0.18), and the best RULER arm at 87.93 still trails FullKV at 88.71 (−0.78), with the residual cost concentrated in multikey_3 (16K: 75–80 versus 89 for TIA), where the 4bit budget granularity dominates on tasks with an extremely large number of distractor keys. On speed, the multi-request batched form still trails dense by 8% (launch-bound), with cross-request batching of forward_extend as the identified fix, and TLI currently ranks third in the three-way indexer latency comparison, with cross-request batched kernelization as the path to cash out. On scale, the subspace mechanism weakens on Qwen3-32B (entry-recall protocol: 0.66–0.87 with a gap of ≤ 0.08 to the full dimension; the mechanism exists, reported as measured), and latent indexing for MLA architectures (the DeepSeek family) is future work. The far statistic of the gate is a single per-layer value, which cross-request contamination affects in mixed batches (production semantics should be per-row).

Appendix A: Index of Experiment Identifiers

The mapping from scripts to conclusions for all experiment identifiers and system milestones (including commit hashes, cross-check protocols, and registered negative results) is maintained in the companion experiment log shipped with the source code.

Appendix B: Reproducibility

2×H20-3e (141GB each), torch 2.8.0+cu128, sglang two-level-indexer branch; RULER official pre-generated set (KVCache-Factory mirror), LongBench v1 full, and the real weights of Qwen3-8B, 30B-A3B, and 32B; string_match_all official scoring; e2e differencing by alternating sparse and dense configurations in matched runs.

References

- [1] DeepSeek-AI. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.14348, 2025.
- [2] Qwen Team. Qwen3 Technical Report. arXiv:2505.09388, 2025.
- [3] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *NeurIPS*, 2022.
- [4] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *SOSP*, 2023.
- [5] Y. Tang, L. Lin, Y. Cao, S. Li, F. Yang, Y. Wang, and T. Wu. Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference. In *ICLR*, 2025.
- [6] Y. Li, Y. Huang, B. Yang, Y. Li, Y. Liu, and M. Qiu. SnapKV: LLM Knows What You Are Looking For Before Generation. In *NeurIPS*, 2024.
- [7] Z. Zhang, Y. Sheng, T. Zhou, D. Chen, L. Zheng, B. Chen, L. Cai, M. Yan, Y. Yu, J. Tang, J. Xu, P. Zhang, and R. Chen. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *NeurIPS*, 2023.
- [8] Z. Cai, Y. Zhang, B. Wang, X. Liu, H. Zhao, Y. Lu, Y. Wang, X. Sun, K. Liu, and Z. Duan. PyramidKV: Dynamic KV Cache Compression Based on Pyramidal Information Funneling. arXiv:2406.02069, 2024.
- [9] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient Streaming Language Models with Attention Sinks. In *ICLR*, 2024.
- [10] I. Beltagy, M. E. Peters, and A. Cohan. Longformer: The Long-Document Transformer. arXiv:1904.07785, 2020.
- [11] M. Zaheer, M. Guruganesh, K. A. Dubey, J. Ainslie, C. Alberti, S. Ontañón, P. Pham, A. Ravula, Q. Wang, L. Yang, and A. Ahmed. Big Bird: Transformers for Longer Sequences. In *NeurIPS*, 2020.
- [12] SparQ Attention: Accelerating LLM Inference with Accurate KV Retrieval, Minimized Decoding Costs, and No Fine-Tuning. arXiv:2406.09050, 2024.
- [13] DeepSeek-AI. DeepSeek-V3.2-Exp Technical Report: Boosting Long-Context Efficiency with Exact Sparse Attention. 2025.
- [14] Moonshot AI. MoBA: Mixture of Block Attention for Efficient Long-Context LLMs. arXiv:2502.13189, 2025.
- [15] HISA: Efficient Hierarchical Indexing for Fine-Grained Sparse Attention. In *COLM*, 2026. arXiv:2603.28458.
- [16] C.-P. Hsieh, S. Sun, S. Krizan, S. Ganti, J. Zhang, B. Kulis, K. Papayiannis, M. Fofanos, N. Rudrajit, and C. Tigges. RULER: What’s the Real Context Size of Your Long-Context Language Models? arXiv:2404.06654, 2024.

- [17] Y. Bai, X. Lv, J. Zhang, H. Zhao, X. Qi, L. Huang, Z. Du, X. Wang, Y. Zheng, Y. Zheng, J. Zhang, Y. Li, T. Su, and J. Tang. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. In *ACL*, 2024.